

# Phân Tích Thuật Toán Bài Toán Aura Tracker

## 1 Phương pháp thiết kế thuật toán

Bài toán được giải bằng kỹ thuật **quét theo trục (Sweep Line)** trên trục  $x$ , kết hợp cùng cấu trúc dữ liệu được sắp xếp để truy vấn bằng **tìm kiếm nhị phân (binary search)**. Ý tưởng chính như sau:

- Mỗi vùng Aura được chuyển thành hai sự kiện: mở vùng tại  $x_1$  và đóng vùng tại  $x_2$ .
- Mỗi hunter được xem như một sự kiện tại vị trí  $x$  của nó.
- Các sự kiện được sắp xếp theo trục  $x$ , sau đó xử lý tuần tự từ trái sang phải.
- Duy trì tập các vùng đang hoạt động (đang mở) được sắp theo  $y_1$  nhằm có thể truy vấn vùng bao chứa hunter thông qua binary search.

## 2 Tại sao phương pháp này phù hợp

Phương pháp sweep line phù hợp vì:

- Ràng buộc lớn ( $n, m \leq 10^5$ ), do đó mọi giải pháp duyệt toàn bộ  $O(nm)$  đều không khả thi.
- Hunter chỉ có thể nằm trong các vùng thỏa  $x_1 \leq x \leq x_2$  tại thời điểm xét, nên quét theo trục  $x$  giúp loại bỏ phần lớn vùng không liên quan.
- Không cần sử dụng các cấu trúc nặng như Segment Tree hay Interval Tree hai chiều.
- Chi phí thao tác thấp:

$$\text{Insert} = O(\log n), \quad \text{Delete} = O(\log n), \quad \text{Query} = O(\log n).$$

## 3 Phân tích độ phức tạp

### 3.1 Độ phức tạp thời gian

- Tạo danh sách sự kiện:  $O(n + m)$
- Sắp xếp sự kiện:  $O((n + m) \log(n + m))$
- Mỗi sự kiện được xử lý trong  $O(\log n)$

Do đó tổng độ phức tạp là:

$$T(n, m) = O((n + m) \log(n + m)).$$

### 3.2 Độ phức tạp không gian

- Lưu các vùng và hunter:  $O(n + m)$
- Tập vùng đang mở:  $O(n)$
- Danh sách sự kiện:  $O(n + m)$

$$\text{Tổng không gian} = O(n + m).$$

## 4 Phân tích chi tiết thuật toán

### 4.1 Sự kiện

Với mỗi vùng  $i$ :

- Sự kiện mở:  $(x_1, 0, i)$
- Sự kiện đóng:  $(x_2, 2, i)$

Với mỗi hunter  $j$ :

$$(x, 1, j)$$

Các sự kiện được sắp theo:

1.  $x$  tăng dần
2. loại sự kiện: mở (0), hunter (1), đóng (2)

## 4.2 Dữ liệu duy trì

- `active_keys`: danh sách  $y_1$  đang mở, được sắp xếp
- `active_ids`: id vùng tương ứng

Khi truy vấn hunter tại  $(x, y)$ , ta dùng `bisect_right` để tìm vùng có  $y_1 \leq y$  gần nhất.

## 5 Code hoàn chỉnh

```
import sys
import bisect

def solve():
    input_data = sys.stdin.read().split()
    it = iter(input_data)

    W = int(next(it))
    H = int(next(it))
    n = int(next(it))
    m = int(next(it))

    rect = []
    for _ in range(n):
        x1 = int(next(it))
        y1 = int(next(it))
        x2 = int(next(it))
        y2 = int(next(it))
        if x1 > x2: x1, x2 = x2, x1
        if y1 > y2: y1, y2 = y2, y1
        rect.append((x1, y1, x2, y2))

    hunters = []
    for _ in range(m):
        x = int(next(it))
        y = int(next(it))
        t = int(next(it))
        hunters.append((x, y, t))

    events = []
    for i, (x1, y1, x2, y2) in enumerate(rect):
        events.append((x1, 0, i))
        events.append((x2, 2, i))
    for j, (x, y, t) in enumerate(hunters):
        events.append((x, 1, j))

    events.sort(key=lambda e: (e[0], e[1]))

    active_keys = []
    active_ids = []

    ans = [0] * n
    freeAura = 0
```

```

for x, typ, idx in events:
    if typ == 0:
        y1 = rect[idx][1]
        pos = bisect.bisect_left(active_keys, y1)
        active_keys.insert(pos, y1)
        active_ids.insert(pos, idx)

    elif typ == 2:
        y1 = rect[idx][1]
        pos = bisect.bisect_left(active_keys, y1)
        while pos < len(active_keys) and active_keys[pos] == y1:
            if active_ids[pos] == idx:
                active_keys.pop(pos)
                active_ids.pop(pos)
                break
            pos += 1

    else:
        xh, yh, t = hunters[idx]
        pos = bisect.bisect_right(active_keys, yh)
        if pos == 0:
            freeAura += t
        else:
            rid = active_ids[pos-1]
            x1, y1, x2, y2 = rect[rid]
            if y1 <= yh <= y2:
                ans[rid] += t
            else:
                freeAura += t

print("␣".join(map(str, ans)), freeAura)

if __name__ == "__main__":
    solve()

```

## 6 Kết luận

Thuật toán sử dụng Sweep Line kết hợp tìm kiếm nhị phân là lựa chọn tối ưu cho bài toán Aura Tracker, đảm bảo tốc độ và hiệu năng tốt với ràng buộc lớn. Với độ phức tạp  $O((n + m) \log(n + m))$ , giải pháp đáp ứng đầy đủ yêu cầu và xử lý hiệu quả số lượng vùng và hunter lớn.